29. a. *Singly Linked List – Destroy List*

–Program Analysis and Time Complexity .

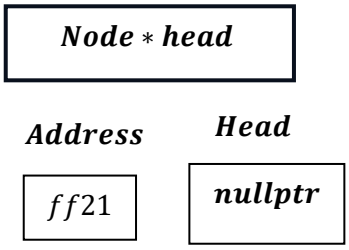
Program: –

```
void destroyList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node **current = head;
    Node *next;
    while (*current != nullptr)
    {
        next = (*current)->next;
        free(*current);
        *current = next;
    }
}
…..
case 27:
        {
            cout << endl;
            cout << endl;
            cout << "Destroying List....." << endl;
            destroyList(&head);
            cout << "List Destroyed." << endl;
            cout << endl;
            cout << endl;
            break;
        }
```

1)*Linked List is Empty*:

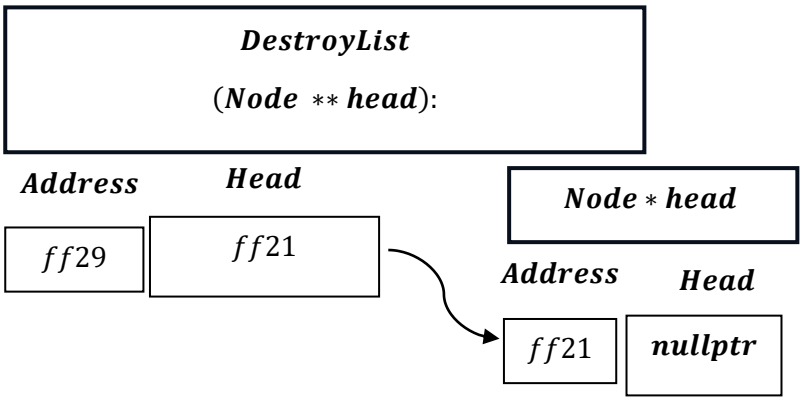
Pictorially depicted as:




```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

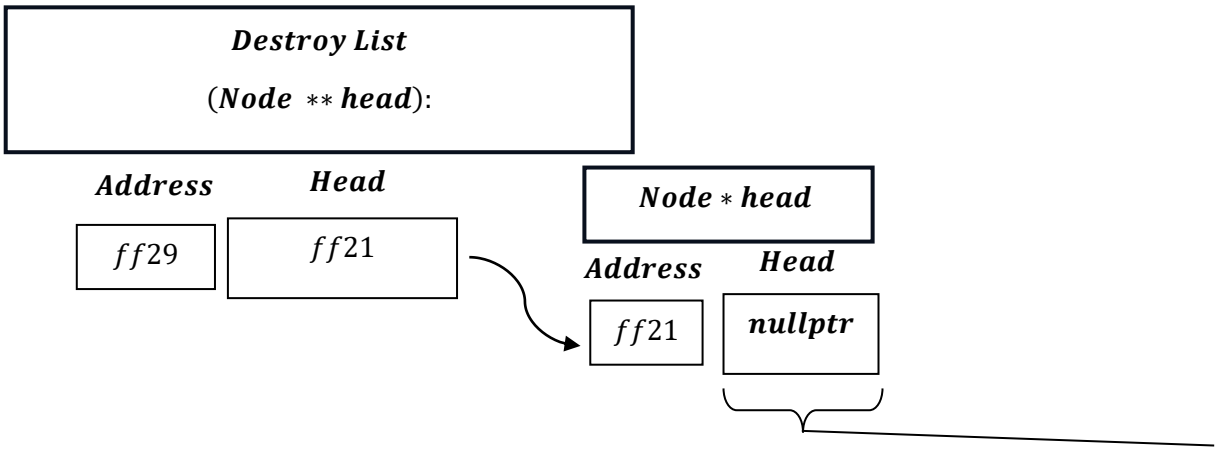
$destroyList(\&head{:}\ ff21);$

```
void destroyList (Node **head) {….}
```



$if(*\ head == nullptr)\ if\ it\ is\ true$

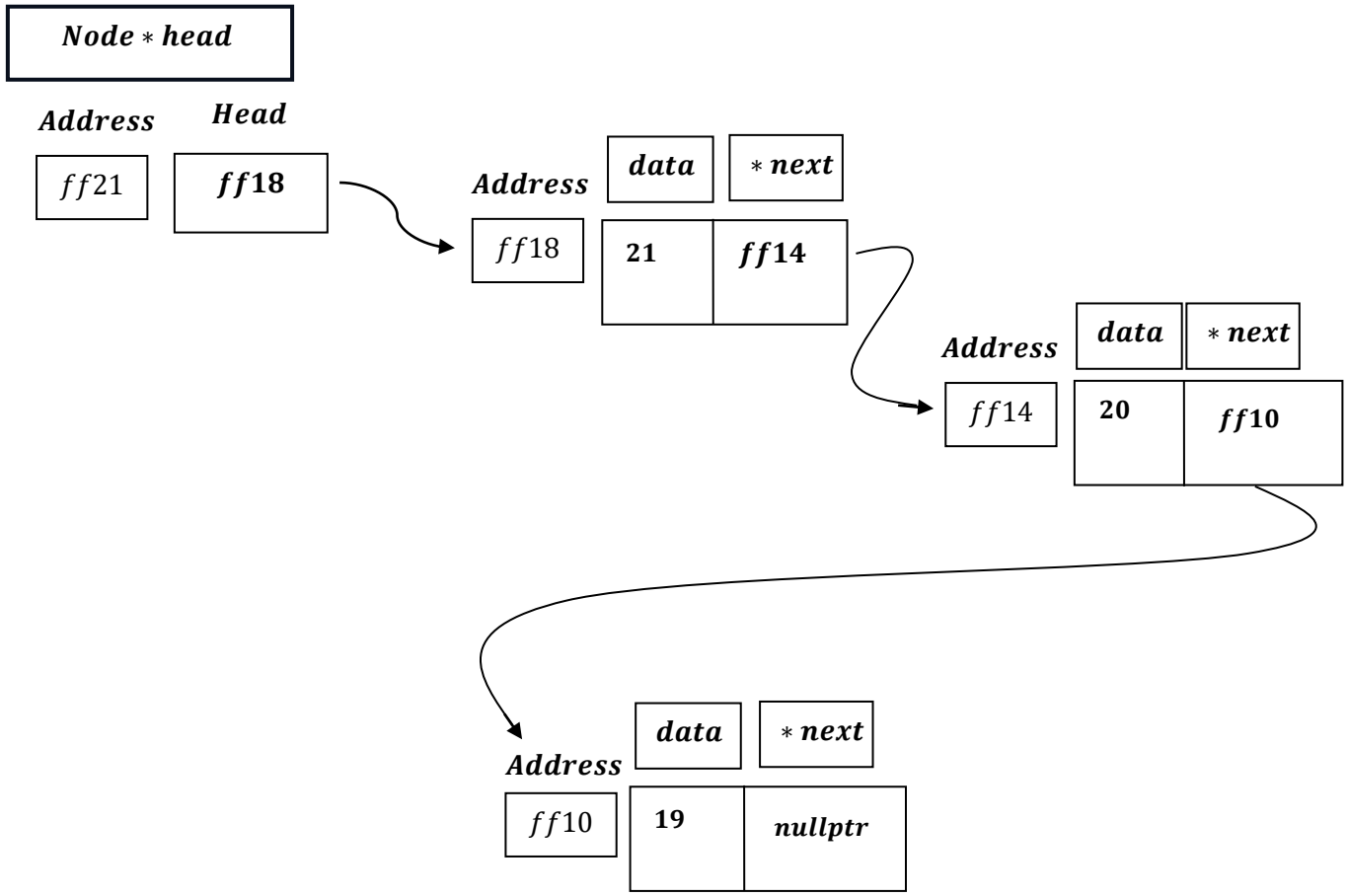
$$\left[\begin{array}{c} \textit{If true then Why true ?} \\ \textit{Ans: Though} * \textit{head which is of Node} ** \textit{head} \\ \textit{temporary pointer to pointer variable of the function} \\ \textit{, as} * \textit{head is used , it dereference from Node} ** \textit{head and} \\ \textit{refer to Node} * \textit{head global pointer to which Node} ** \textit{head} \\ \textit{is pointing to and then its value is checked is} \\ \textit{whether equal to nullptr or not ? This occurs during execution} \\ \textit{and compilation of the `Destroy List` function.} \end{array}\right]$$



If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

---

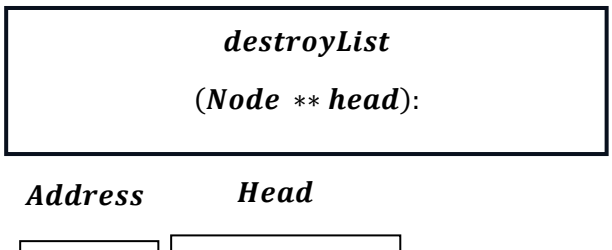
2)*Linked List is not Empty*:

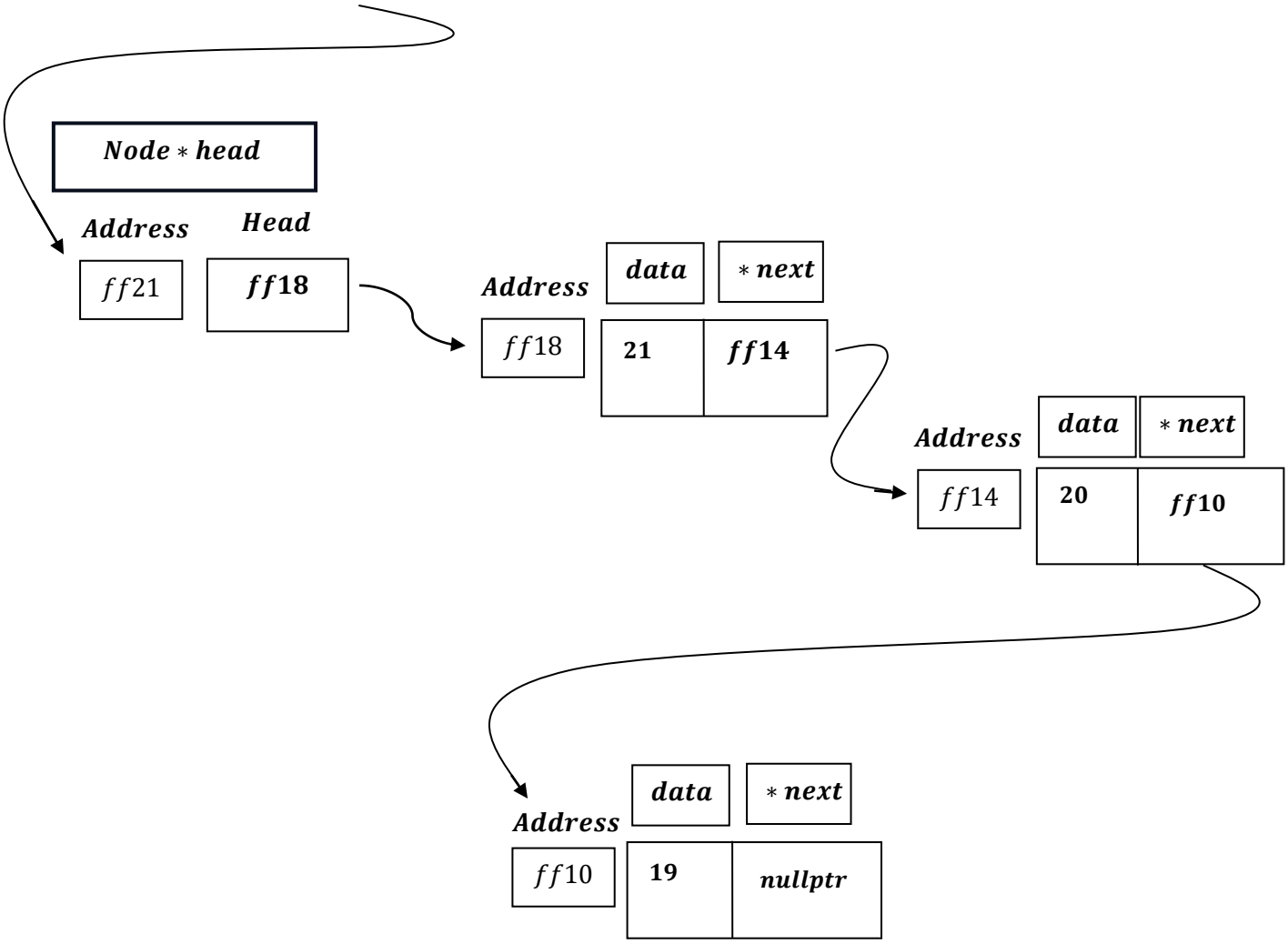


Function Call:

```
destroyList (&head);
```

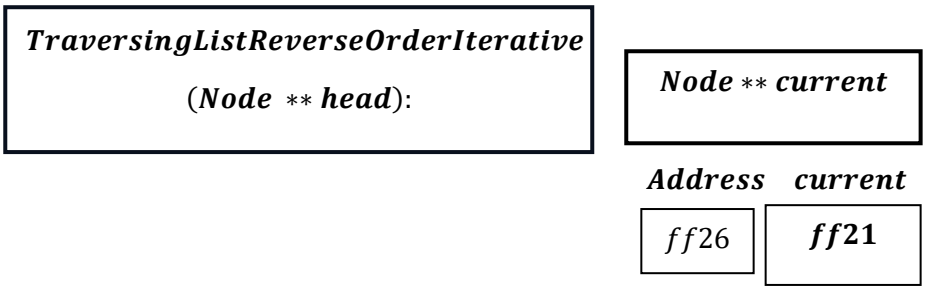
$i.e.\ destoryList(\&head{:}\ ff21)$

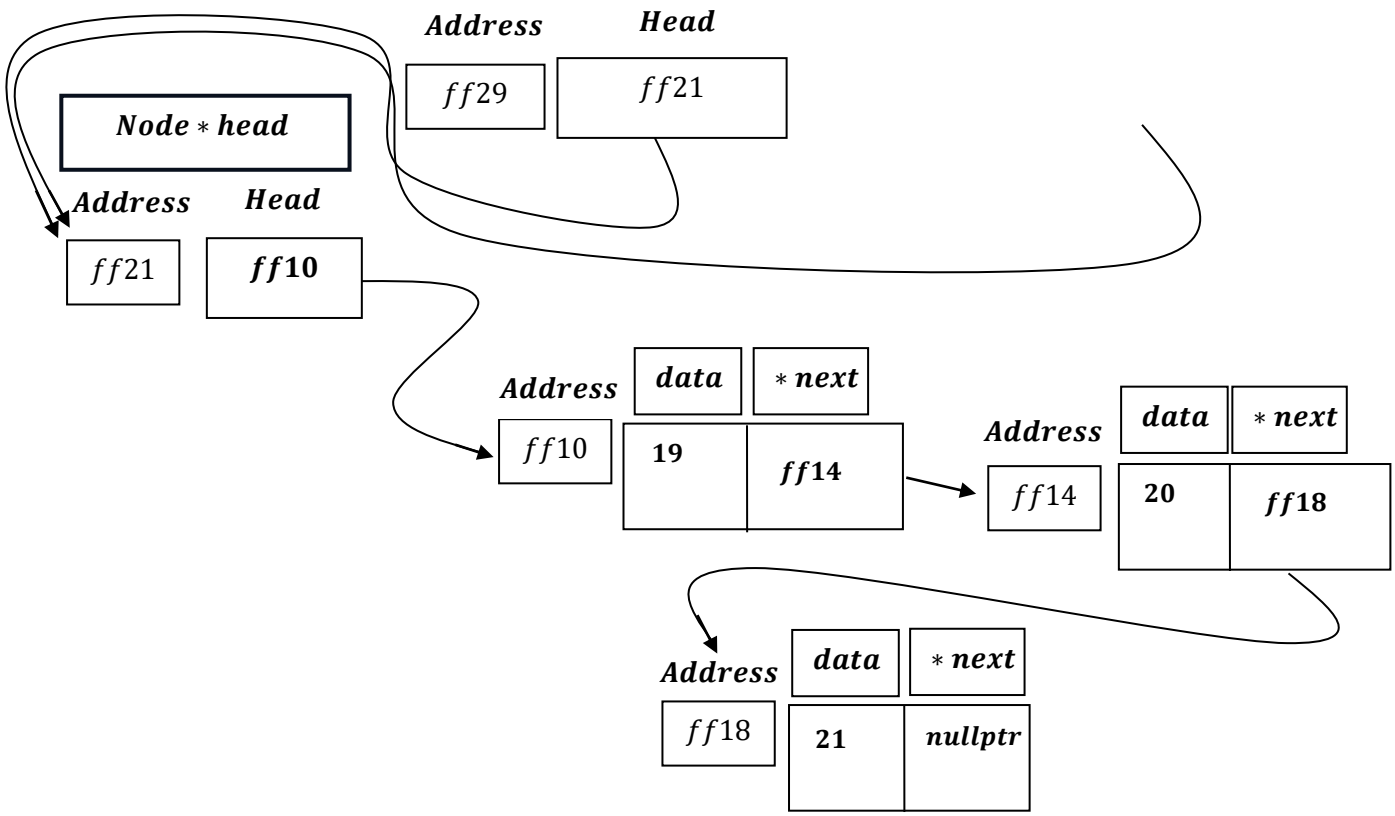





```
    Node **current = head;
```

$Node ** current = head = ff18;$

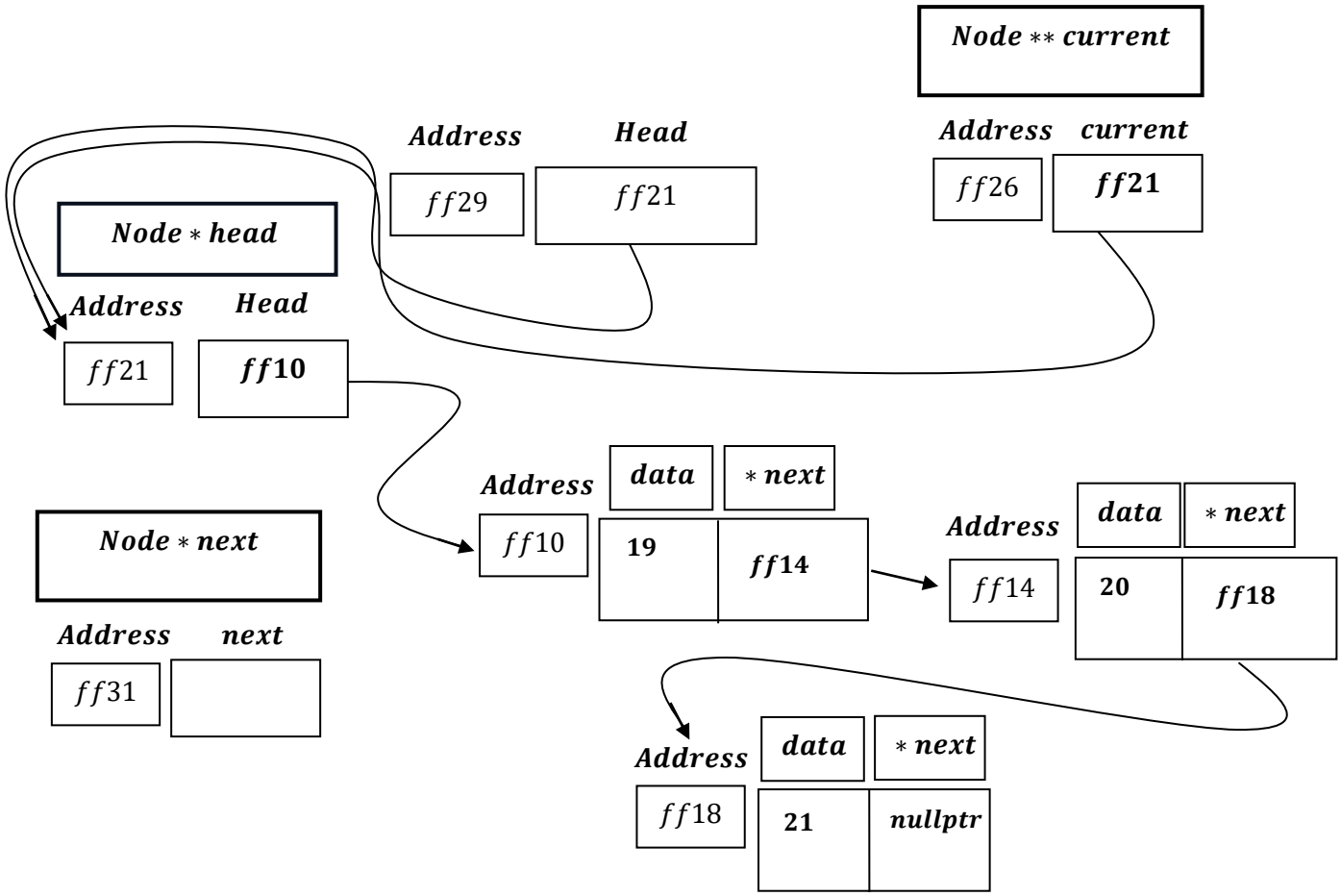





```
    Node *next;
```

$TraversingListReverseOrderIterative$

$(Node\ **\ head)$:




```
while (*current != nullptr)
 {
     next = (*current)->next;
     free(*current);
     *current = next;
 }
```

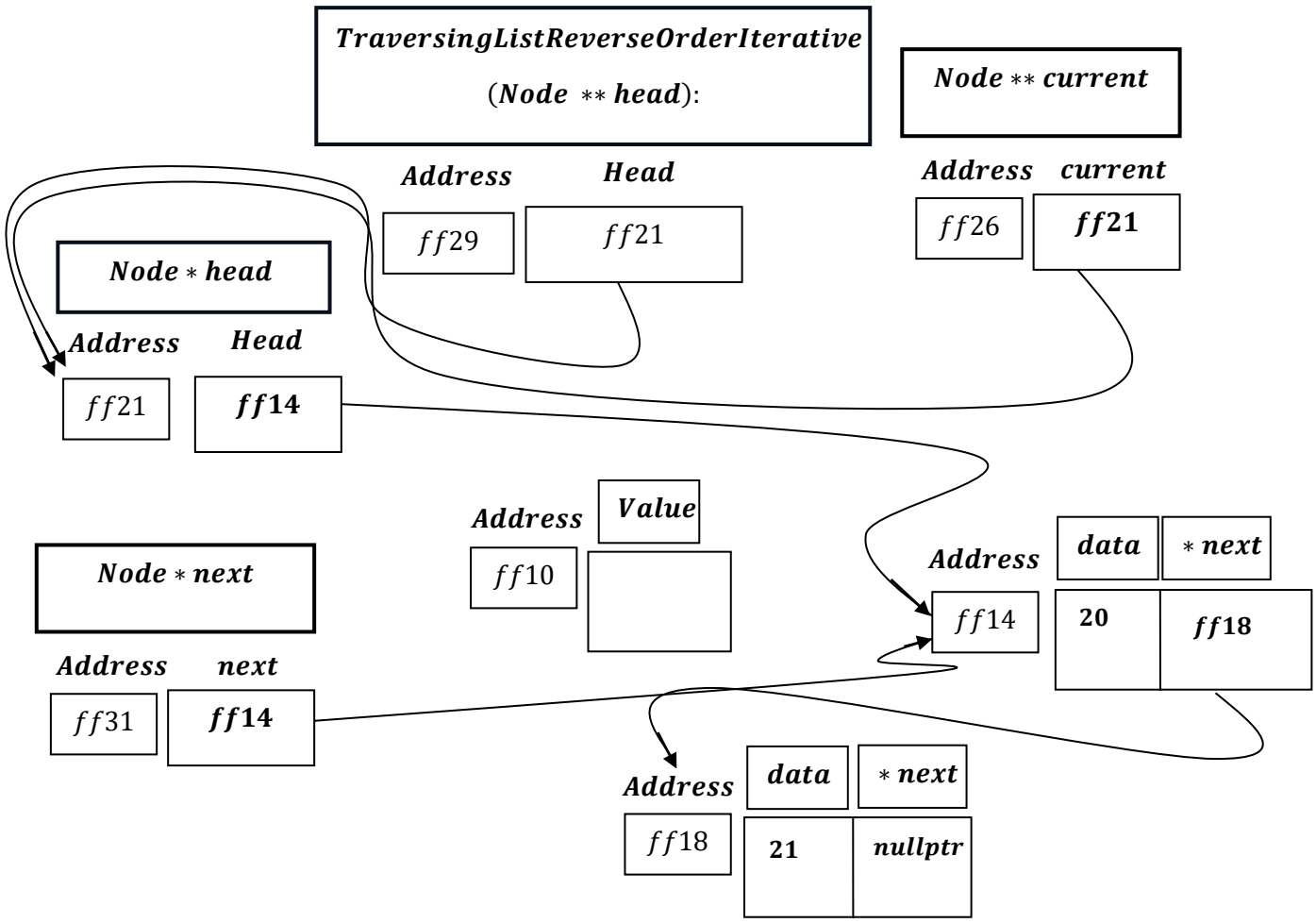
$*current = ff10 \neq NULLPTR$ *is true*, $\left[\begin{array}{c} current = ff21 \text{ and } *current \\ deference \text{ to } ff10. \end{array}\right]$ *hence*:

$next = (*current) \rightarrow next = (ff10 \rightarrow next) = ff14;$

$free(*current);$

$*current = next = ff14;$

$\left[\begin{array}{c} Note: Node ** current \text{ will continue to point to Head but Head will now point} \\ \text{to next Node } i.e. ff14. \end{array}\right]$



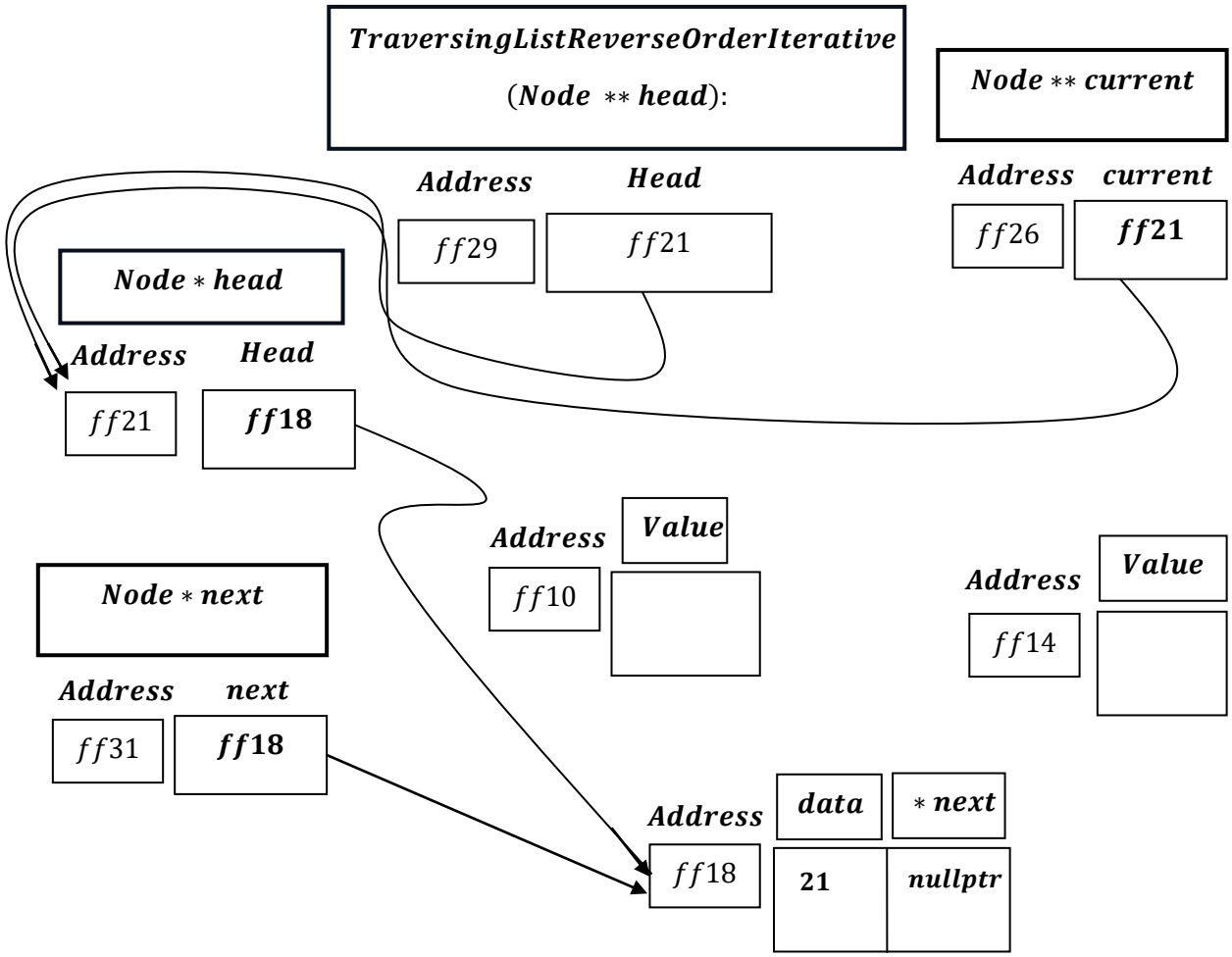
$$*\,current = ff14 \neq NULLPTR\ is\ true\ , \begin{bmatrix} current = ff21\ and\ *\,current \\ deference\ to\ ff14. \end{bmatrix}\ hence:$$

$next = (*\,current) \rightarrow next = (ff14 \rightarrow next) = ff18;$

$free(*\,current);$

$*\,current = next = ff18;$

$$\begin{bmatrix} Note: Node\ **\ current\ will\ continue\ to\ point\ to\ Head\ but\ Head\ will\ now\ point \\ to\ next\ Node\ i.e.\ ff18. \end{bmatrix}$$



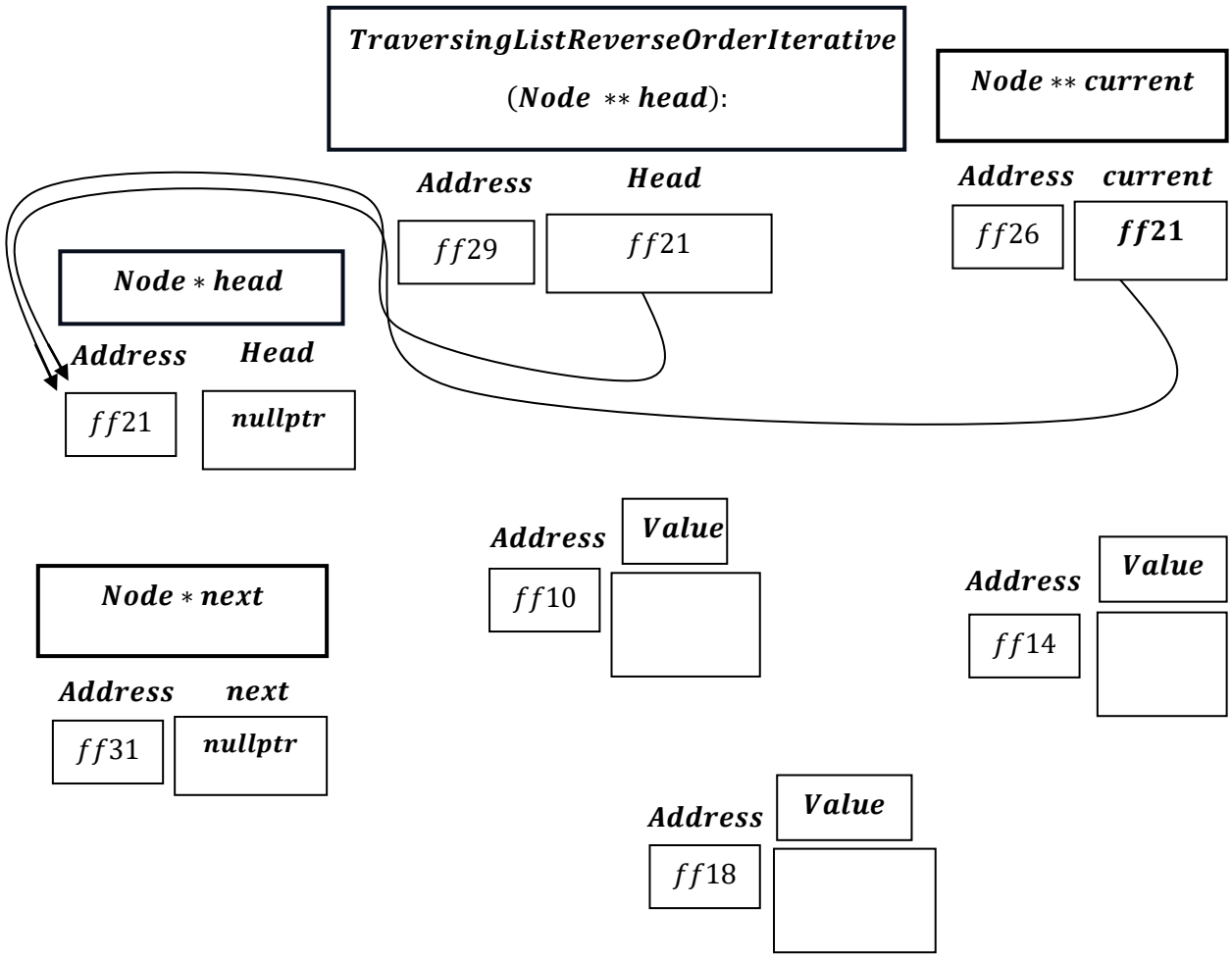
$*current = ff18 \neq NULLPTR$ is true, $\left[\begin{array}{c} current = ff21 \text{ and } *current \\ deference \text{ to } ff18. \end{array}\right]$ hence:

$next = (*current) \rightarrow next = (ff18 \rightarrow next) = NULLPTR;$

$free(*current);$

$*current = next = ff18;$

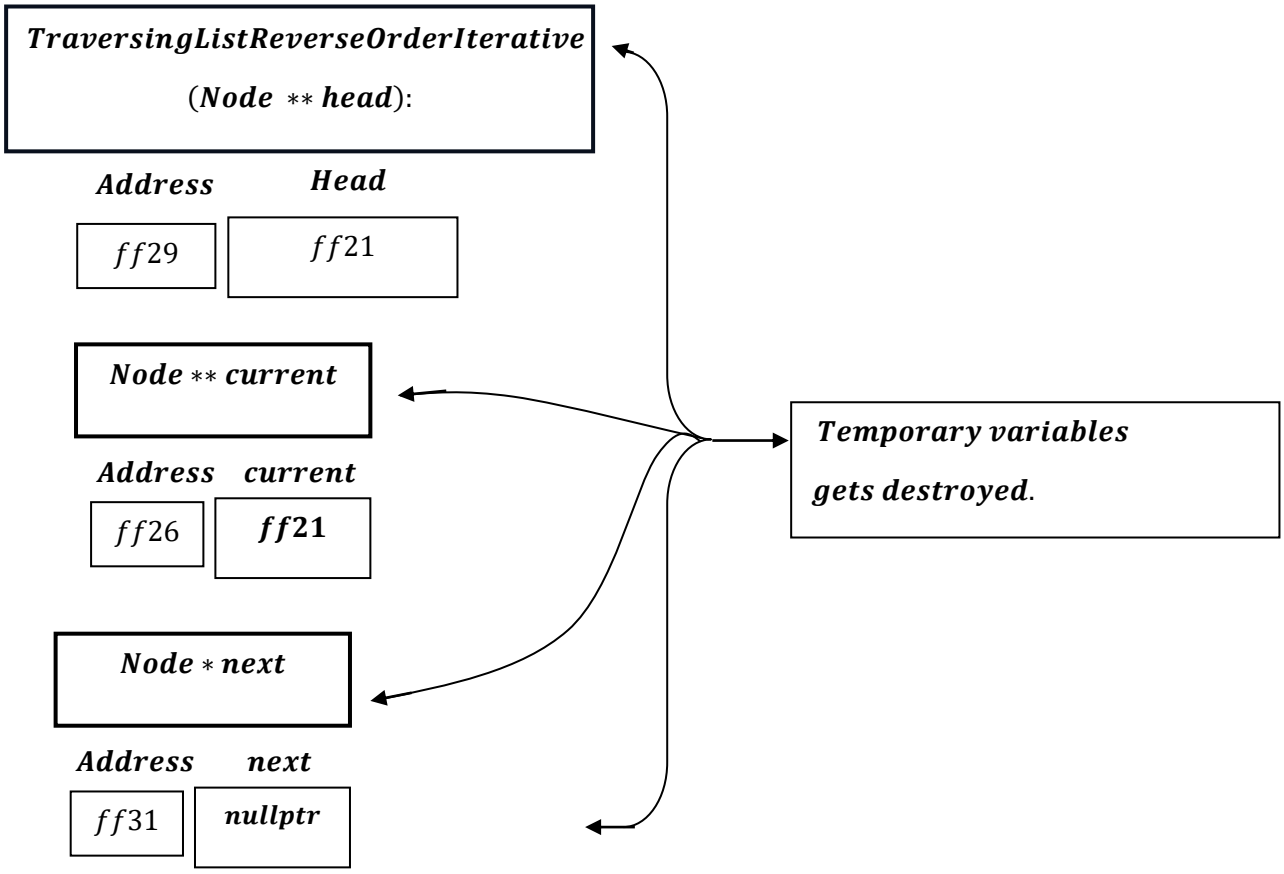
$\left[\begin{array}{c} Note: Node ** current \text{ will continue to point to Head but Head will now point} \\ \text{to } NULLPTR. \end{array}\right]$



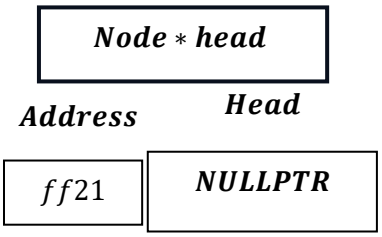
$*current = NULLPTR \neq NULLPTR$ is false , hence, exit while loop.

As function ends , Node ** Head , Node ** current and

Node * next local variable pointers, gets detroyed:



And we are left with:



Singly Linked List – Destroy List [Working Summary]:

This C++ function, `destroyList`, is a robust method for deallocating all memory used by a linked list, ensuring the original `head` pointer is safely updated. It's a bit more advanced because it uses a pointer to a pointer (`Node **head`).

Working Summary

1. **Handles Empty List:** The function first checks if the list is empty by dereferencing the `head` pointer (`*head == nullptr`). If the list is empty, it prints a message and exits.
2. **Uses a Pointer-to-Pointer:** It accepts a pointer to the head pointer (`Node **head`). This allows the function to **directly modify the original `head` pointer** in the calling function (e.g., in `main`), not just a copy of it.
3. **Initializes for Traversal:** It creates a temporary pointer `next` to hold the address of the next node during deletion.
4. **Iterates and Deletes:** The function enters a `while` loop that continues as long as the list is not empty (`*current != nullptr`). In each step:
    - It **saves the address** of the next node (`next = (*current)->next`).
    - It **frees the memory** of the current node (`free(*current)`).
    - It then **updates the actual `head` pointer** to point to the next node (`*current = next`). As the loop progresses, this effectively moves the `head` pointer down the list, deleting the node behind it until the entire list is gone and `head` becomes `nullptr`.

By the end of the loop, every node has been freed, and the original `head` pointer that was passed into the function is now set to `nullptr`, completely and safely destroying the list.

Time Complexity Of Destroy List Function Module

```
void destroyList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node **current = head;

    Node *next;


    while (*current != nullptr)
    {


        next = (*current)->next;

        free(*current);

         *current = next;
    }
}
```

$O(1)$

$O(1)$

$O(1)$

RUNS `N` times WORST TIME COMPLEXITY.

$1 + 1 + 1 + \cdots n\ times = O(n)$

$1 + 1 + 1 + \cdots n\ times = O(n)$

$1 + 1 + 1 + \cdots n\ times = O(n)$

Program Analysis

Absolute Best Case:

$When\ List\ is\ Empty\ (head = NULLPTR)$

$\rightarrow If\ head = nullptr$ *, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes,* $O(1)$ *time.*

$\underline{Worst\ Case(When\ \ List\ is\ not\ Empty)}$

$\rightarrow Node ** current = \ head; this\ assignment\ takes\ constant\ time\ i.e. O(1).$

$\rightarrow Declaration\ of\ Node * next\ ; takes\ constant\ time\ i.e. O(1).$

$\rightarrow while(* current\ \ != nullptr)\ checking\ becomes\ true\ , then:$

$\rightarrow next = (* current) \rightarrow next; such\ assignment\ takes, 1 + 1 + \cdots n\ times = n;$

$\rightarrow free(* current)\ ; freeing\ memory\ pointed\ by\ current\ pointer;$

$1 + 1 + \cdots n\ times = n;$

$\rightarrow * current = next; such\ assignment\ takes, 1 + 1 + \cdots n\ times = n;$

$Therefore\ , total\ loop\ time\ complexity: n + n + n = 3n = 3 \times O(n) = O(n)\ .$

$Hence\ total\ time\ complexity\ : O(1) + O(1) + O(n) = O(n).$

$Therefore\ takes\ O(n)\ time\ .$

$Average\ Case$

$We\ can\ say\ , if\ List\ is\ Empty\ , we\ get\ our\ best\ case\ scenario, and$
$when\ list\ is\ not\ empty\ , we\ get\ our\ worst\ case\ scenario. But\ though$
$best\ case\ and\ worst\ case\ scenarios\ are\ present\ as, always\ we$
$can\ determine\ list\ get\ traversed\ from\ beginning\ to\ end\ , hence\ \ we\ can\ say$
$average\ case\ is\ our\ worst\ case\ here\ i.e. always\ the\ entire\ list\ is\ traversed$
$i.e.\ \ O(n) = \Theta(n)\ .$

$This\ means\ the\ execution\ time\ is\ solely\ dependent\ on\ the\ length\ of\ the\ list.$
$If\ the\ list\ is\ empty, the\ execution\ time\ is\ constant\ (best\ case).$

$If\ the\ list\ is\ not\ empty, the\ execution\ time\ is\ proportional\ to\ the\ length\ of\ the\ list\ (worst\ case).$

$There's\ no\ element\ of\ randomness\ or\ varying\ input\ data\ to\ consider.$
$Therefore, the\ concept\ of\ an\ average\ case, where\ we\ average\ the\ execution\ time\ across\ different\ input\ possibilities, doesn't\ strictly\ apply\ here.$
$Hence\ , Destroy\ List\ function\ doesn't\ have\ a\ true\ average\ case.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq N \leq c_2 \times N$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$Destroy\ List$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case\ (= Worst\ Case)$	$\Theta(n)$